Stat405 Final Project
Aiwei Yang 42274209
Chang Sun 19062439

Aiwei Yang primarily focused on data processing, EDA and visualization. Chang Sun was mainly responsible for model implementation in Stan including both HMC and VI methods. We jointly contributed to model design, final interpretation and writing the final report.

Part 1. Introduction

Bicycle renting systems are one of the popular modes of transportation used by individuals in urban areas owing to the convenience and environmental benefits involved. Therefore, there is a need to have adequate knowledge about the various determinants of bicycle rental demand.

In the current research paper, a time series dataset is analyzed where the independent variables include weather and calendar related variables, while the dependent variable involves the number of bicycles rented.

This paper aims to identify and quantify the effects of weather and calendar-related factors on bike rental demand in the U.S. for the period 2018-2019.

Part 2. Data Description & Exploratory Analysis

Data Description:

This dataset contains daily data of bike rental demand with several variables that explain factors like weather and calendars. The dependent variable is total bike rentals (cnt), which shows daily demand for bikes.

The independent variables are categorized into three groups. These include weather variables temp (temperature), hum (humidity), and windspeed. The calendar variables include season, which indicates different seasons, workingday, which is an indicator of whether it is a workday or not, and holiday, which determines whether it is a holiday or not. yr is the variable that explains the trend over time.

Some variables are not considered in this study. The variable instant serves as an index, and thus, it cannot predict anything. Casual and registered variables are ignored since they sum up to the dependent variable cnt, hence violating the assumption of no perfect multicollinearity. Finally, between weekday and workingday, workingday is considered due to its simplicity.

Research Question:

Which weather and calendar-related factors are associated with daily bike rental demand?

Modelling Challenge:

First, the dependent variable is daily bicycle rental demand (*cnt*). It is a non-negative integer and falls under the category of count data. Hence, traditional linear regression analysis is not applicable since special models should be employed.

However, the problem becomes more complex due to variations in the data. In most cases, there will be more variation than expected based on the average demand value. For instance, the variance may be greater than the mean value. This situation leads to overdispersion of the data, which is contrary to the assumptions of the Poisson distribution model.

Thus, alternative approaches like using the negative binomial regression model are recommended. Another benefit of Bayesian methods is that uncertainties associated with the parameter estimates can be directly calculated.

Exploratory Data Analysis:

The daily number of bicycles rented covers a wide range, with most observations taking place in the interval between 2,000 and 7,000. Extreme values are not as common but still noticeable. Variability from day to day is quite considerable. There is a clearly visible positive correlation between temperature and rentals; higher temperatures correlate with greater demand, although there is still considerable dispersion among the data points. Humidity demonstrates a slight negative correlation with the number of rentals; demand drops as humidity grows, although the correlation is relatively weak. Wind speed correlates negatively with demand; as the wind gets stronger, the number of rentals decreases slightly. However, there is some variability present. Seasonal differences have proved to be rather significant. The demand is lower during the spring season, higher during the summer and autumn seasons, and more variable during the winter one. Working days and weekends demonstrate only minor differences; their distributions are quite similar. By comparison, holidays are characterized by lower demands, which can be seen by the overall drop in the distribution's position. Finally, there is a much higher demand in 2019 than in 2018. These patterns provide initial evidence of overdispersion and nonlinear variability in the data, motivating the use of more flexible count models such as the Negative Binomial specification.

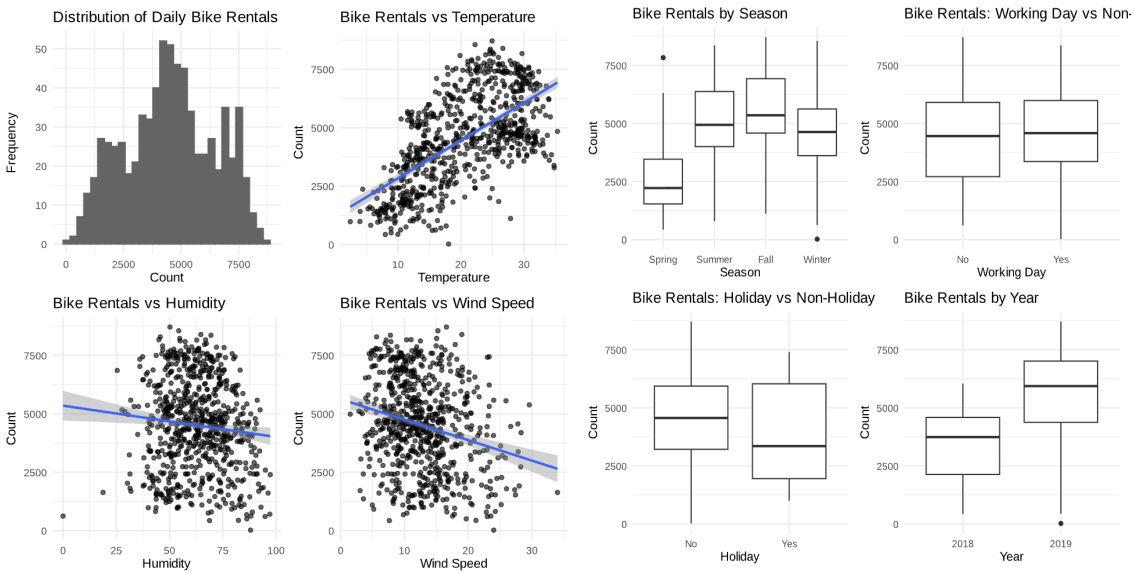


Figure 1. EDA of selected variables

Model 1: Bayesian Poisson Regression Model

(1) Likelihood

Let cnt_i denote the number of bike rentals on day i. Since the response variable is count-valued and non-negative, we model it using a Poisson distribution:

$cnt_i \sim Poisson(\lambda_i)$

where the mean parameter λ_i is linked to the covariates through a log-linear model:

$log(\lambda_i) = X_i\beta$

Here, X_i represents the vector of explanatory variables, including weather and calendar-related features.

(2) Prior

To complete the Bayesian specification, we assign independent priors to the regression coefficients:

$\beta_j \sim N(0, 10)$

For the regression coefficients, we use normal priors centered at zero with moderate variance. This reflects the lack of strong prior knowledge while preventing unrealistically large effects from any single variable.

(3) Why Poisson

The Poisson model provides a natural starting point for modeling count data, as it directly captures the non-negative and discrete nature of the response variable. It also serves as a baseline model for comparison with more flexible alternatives, such as the Negative Binomial model.

Model 2: Bayesian Negative Binomial Regression

(1) Likelihood

Let cnt_i denote the total bike rental demand on day i. To allow for overdispersion in the count outcome, we consider a Negative Binomial model:

$cnt_i \sim NegBin(\mu_i, \phi)$

with mean parameter linked to the covariates through

$log(\mu_i) = X_i\beta$

where, X_i contains the selected weather and calendar-related variables, and is a dispersion parameter.

(2) Prior

We assign the same weakly informative priors to the regression coefficients:

$\beta_j \sim N(0, 10)$

and place a positive prior on the dispersion parameter:

$\phi \sim Gamma(0, 1)$

For the dispersion parameter in the Negative Binomial model, we impose a prior that ensures positivity and avoids extreme values. This helps stabilize estimation while still allowing the data to determine the level of overdispersion. While the prior on ϕ ensures positivity and regularizes the dispersion estimate.

(3) Why Negative Binomial

The Poisson model assumes that the conditional variance is equal to the conditional mean. For bike rental demand, this assumption may be too restrictive, since daily demand can vary substantially across days even under similar weather and calendar conditions. In such cases, the variance may exceed the mean, indicating overdispersion.

The Negative Binomial model relaxes this restriction by introducing an additional dispersion parameter. This makes it more flexible than the Poisson model and often more suitable for count data with extra variability.

(4) Comparison with Model 1:

This model extends the Bayesian Poisson regression by allowing the variance to exceed the mean. It is therefore expected to provide a better fit when overdispersion is present in the data. In the later model

comparison section, we compare the Poisson and Negative Binomial specifications to assess whether the additional flexibility leads to improved fit and predictive performance.

Literature review

A highly relevant study conducted by Xu (2024) analyses the same set of U.S. bike sharing data from 2018 to 2019 and uses multiple linear regression for daily demand prediction. The procedure includes data preparation, creation of dummy variables for categorical data, and using various methods, including variance inflation factor (VIF) and p-values for predictor selection. As a result, such indicators as temperature, year, and weather significantly affect bike demand.

On the contrary, the current research uses a Bayesian approach and applies Poisson and Negative Binomial regression due to the fact that the data belongs to counts. In addition, uncertainty analysis based on posterior distributions will be emphasized, and several regression models will be utilized in order to overcome overdispersion.

Posterior Inference Methods

HMC is used as the primary inference method, as it provides asymptotically exact samples from the posterior distribution and allows reliable uncertainty quantification.

VI is used as a faster approximation method. While it is computationally efficient, it may underestimate posterior variance and can introduce bias in parameter estimates. Comparing the two methods allows us to evaluate the trade-off between computational speed and inferential accuracy.

HMC: Hamiltonian Monte Carlo is the main technique used for estimating the posterior distribution. Hamiltonian Monte Carlo uses gradient-based proposal distributions to create posterior samples, and it is considered a better technique for conducting inference in this study. The posterior convergence for the Poisson distribution involves the values of 1.003341, 1529.346, and 1831.3, which are the highest value of $\widehat{R}$, lowest bulk effective sample size, and lowest tail effective sample size, respectively. Similarly, the posterior convergence for the negative binomial distribution involves the values of 1.004275, 1899.109, and 2758.454.

VI: The use of Variational Inference (VI) acts as a quicker approximate approach. In the case of the Poisson distribution, median ELBO convergence occurred after 3,700 iterations. Median ELBO convergence occurred after 1,300 iterations for the Negative Binomial distribution. VI is notably faster compared to HMC; however, it is only an approximation and not the inference solution itself.

Posterior Results and Converge Diagnostic

Both posterior calculations show consistency overall, and HMC diagnostic results support the usage of the posterior sample for analysis purposes. Four chains used in Poisson converge without any difficulties. The highest value of the Gelman-Rubin statistic is 1.003341, whereas the lowest bulk and tail effective sample sizes are 1529.346 and 1831.3, respectively. For the negative binomial model, the metrics equal 1.004275, 1899.109, and 2758.454. Such results indicate appropriate mixing of chains, which means that the posterior sample size is more than enough to capture the parameter set.

Although the Negative Binomial fit reports warnings during warmup, these occur sporadically at early proposals and do not persist in the final posterior diagnostics. Since the final $\widehat{R}$ values remain close to 1 and the effective sample sizes are large, the final HMC fit is still judged to be reliable.

To illustrate mixing directly, the traceplots for selected coefficients and the dispersion parameter from the Negative Binomial model are shown below. The traceplots show good

mixing across chains with no visible trends, indicating that the HMC sampler has converged and is exploring the posterior distribution effectively.

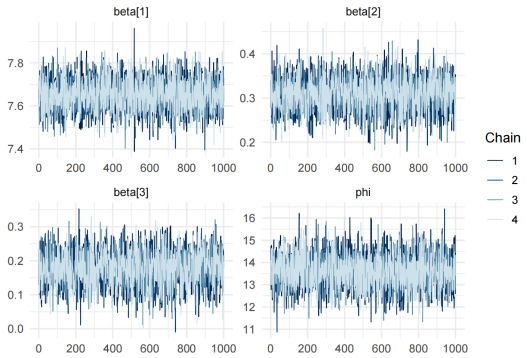
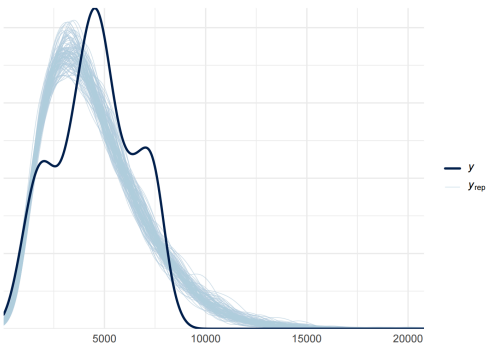


Figure 2. Traceplots for selected parameters under the Negative Binomial HMC fit

Posterior Predictive Checks and Model Fit

Posterior predictive checking methods are used to determine how well the models fit the distribution of bike rentals per day. Both models have an accurate representation for the mean. The mean of the observations is 4508.007; on the other hand, it is 4508.043 for the Poisson model and 4555.502 for the Negative Binomial model. The main difference between the models and the observations is in variance. The observed variance is 3,748,141; however, for the Poisson model it is 3,143,307, and for the Negative Binomial model – 5,731,131.



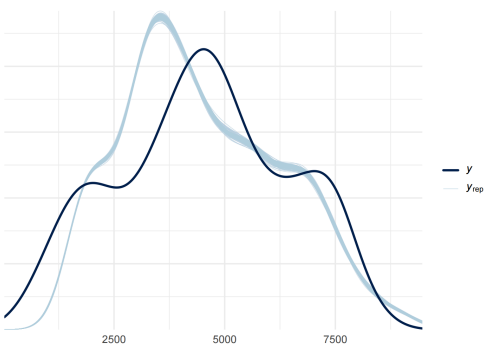


Figure 3. Posterior predictive density overlay under the Poisson model Figure 4. Posterior predictive density overlay under the NB model

The posterior predictive plots highlight clear differences between the two models. Under the Poisson specification, the replicated distributions display a much heavier right tail and a wider spread than the observed data. The simulated densities extend far beyond the range of the empirical distribution, indicating that the model generates unrealistically large values. In addition, the overall shape of the predictive distribution does not align well with the observed central mass. This suggests that the Poisson model fails to capture both the variability and the distributional shape of bike rental demand.In contrast, the Negative Binomial model produces predictive distributions that are much closer to the observed data. The central mass, overall shape, and range of the replicated distributions align more closely with the empirical distribution. The model is able to better accommodate the variability in the data, particularly

by reducing the excessively long tail observed under the Poisson model. However, some discrepancies remain, especially in the tails, where the model still slightly overestimates variability.

Overall, these posterior predictive checks indicate that the Poisson model provides a poor fit to the data, while the Negative Binomial model offers a substantial improvement by accounting for overdispersion, although it does not perfectly reproduce all features of the observed distribution.

Comparison of HMC and VI, Limitations and Conclusions

The comparison between HMC and VI demonstrates that the latter produces a considerably shorter calculation time regardless of the selected specification. In particular, VI requires 0.763 seconds for the Poisson model and 0.965 seconds for the Negative Binomial, while HMC consumes 1.809 and 6.043 seconds, correspondingly. At the same time, VI produces smaller posterior standard deviation values, indicating possible underestimation of posterior uncertainty. Although posterior means appear to be roughly equal, HMC seems to be more advantageous since it provides a full posterior sampling as opposed to VI.

Posterior predictive checks have been used to examine the adequacy of the fitted models. As can be seen from Figure 3 below, the Poisson model cannot explain variability and shapes of the data, while the Negative Binomial model explains the data significantly better due to consideration of overdispersion. However, it does not perfectly match the distribution of the observed data.

It is necessary to note several caveats of this analysis. First of all, there can be no causality inferred from the results. Secondly, some relevant variables have not been considered. Finally, despite a relatively good match, the Negative Binomial model still does not capture the data-generating process in its entirety since posterior predictive checks reveal some residual differences.

Overall, one can state that taking into account overdispersion when developing models for analyzing count data is crucial to avoid incorrect conclusions and unrealizable prediction. The use of the Bayesian approach has proved to be successful; moreover, the Negative Binomial model captures data much better than the Poisson model.

Stat405_coding

April 17, 2026

0.0.1 Stat405 Final Project: Coding Part

1. Clean Data

[30]:
```
install.packages("janitor")
install.packages("patchwork")
# Load required libraries
library(tidyverse)
library(janitor)

# 1. Load the dataset and clean column names
bike <- read_csv("day2018.csv") %>%
  clean_names()

# 2. Inspect the structure and summary of the dataset
glimpse(bike)
summary(bike)

# 3. Check for missing values in each column
colSums(is.na(bike))

# 4. Select only the variables used in the analysis
bike_clean <- bike %>%
  select(dteday, cnt, season, yr, hum, windspeed, workingday, holiday, temp)

# 5. Convert date column to Date format
bike_clean <- bike_clean %>%
  mutate(dteday = as.Date(dteday, format = "%d-%m-%Y"))

# 6. Convert categorical variables to factors with meaningful labels
bike_clean <- bike_clean %>%
  mutate(
    season = factor(
      season,
      levels = c(1, 2, 3, 4),
      labels = c("Spring", "Summer", "Fall", "Winter")
    ),
    yr = factor(
      yr,
```

```r
      levels = c(0, 1),
      labels = c("2018", "2019")
    ),
    workingday = factor(
      workingday,
      levels = c(0, 1),
      labels = c("No", "Yes")
    ),
    holiday = factor(
      holiday,
      levels = c(0, 1),
      labels = c("No", "Yes")
    )
  )

# 7. Re-check the structure after cleaning
glimpse(bike_clean)
summary(bike_clean)

# 8. Check for duplicate dates
sum(duplicated(bike_clean$dteday))

# 9. Examine the distribution of the response variable
summary(bike_clean$cnt)

# 10. Save the cleaned dataset for further analysis
write_csv(bike_clean, "bike_clean.csv")
```

```
Updating HTML index of packages in '.Library'

Making 'packages.html' …
 done

Updating HTML index of packages in '.Library'

Making 'packages.html' …
 done

Rows: 730 Columns: 16
  Column specification

Delimiter: ","
chr  (1): dteday
dbl (15): instant, season, yr, mnth, holiday, weekday, workingday,
weathersi…

 Use `spec()` to retrieve the full column specification for this
data.
```

```
 Specify the column types or set `show_col_types = FALSE` to quiet
this message.

Rows: 730
Columns: 16
$ instant    <dbl> 1, 2, 3,

4, 5, 6, 7, 8, 9,

10, 11, 12, 13, 14,

15, 16, 17, …
$ dteday     <chr> "01-01-2018", "02-01-2018",

"03-01-2018", "04-01-2018", "05…
$ season     <dbl> 1, 1, 1,

1, 1, 1, 1, 1, 1,

1, 1, 1, 1, 1, 1,

1, 1, 1, 1, 1,…
$ yr         <dbl> 0, 0, 0,

0, 0, 0, 0, 0, 0,

0, 0, 0, 0, 0, 0,

0, 0, 0, 0, 0,…
$ mnth       <dbl> 1, 1, 1,

1, 1, 1, 1, 1, 1,

1, 1, 1, 1, 1, 1,

1, 1, 1, 1, 1,…
$ holiday    <dbl> 0, 0, 0,

0, 0, 0, 0, 0, 0,

0, 0, 0, 0, 0, 0,

0, 1, 0, 0, 0,…
$ weekday    <dbl> 6, 0, 1,

2, 3, 4, 5, 6, 0,

1, 2, 3, 4, 5, 6,

0, 1, 2, 3, 4,…
$ workingday <dbl> 0, 0, 1,

1, 1, 1, 1, 0, 0,

1, 1, 1, 1, 1, 0,

0, 0, 1, 1, 1,…
```

```
$ weathersit <dbl> 2, 2, 1,

1, 1, 1, 2, 2, 1,

1, 2, 1, 1, 1, 2,

1, 2, 2, 2, 2,…
$ temp       <dbl> 14.110847, 14.902598,

8.050924, 8.200000, 9.305237, 8.37826…
$ atemp      <dbl> 18.181250, 17.686950,

9.470250, 10.606100, 11.463500, 11.66…
$ hum        <dbl> 80.5833, 69.6087,

43.7273, 59.0435, 43.6957, 51.8261,

49.86…
$ windspeed  <dbl> 10.749882, 16.652113,

16.636703, 10.739832, 12.522300, 6.00…
$ casual     <dbl> 331, 131, 120,

108, 82, 88, 148, 68,

54, 41, 43, 25, 38, 54…
$ registered <dbl> 654, 670, 1229,

1454, 1518, 1518, 1362, 891,

768, 1280, 122…
$ cnt        <dbl> 985, 801, 1349,

1562, 1600, 1606, 1510, 959,

822, 1321, 126…

    instant         dteday              season            yr
 Min.   :  1.0   Length:730         Min.   :1.000   Min.   :0.0
 1st Qu.:183.2   Class :character   1st Qu.:2.000   1st Qu.:0.0
 Median :365.5   Mode  :character   Median :3.000   Median :0.5
 Mean   :365.5                      Mean   :2.499   Mean   :0.5
 3rd Qu.:547.8                      3rd Qu.:3.000   3rd Qu.:1.0
 Max.   :730.0                      Max.   :4.000   Max.   :1.0
      mnth           holiday           weekday        workingday
 Min.   : 1.000   Min.   :0.00000   Min.   :0.000   Min.   :0.0000
 1st Qu.: 4.000   1st Qu.:0.00000   1st Qu.:1.000   1st Qu.:0.0000
 Median : 7.000   Median :0.00000   Median :3.000   Median :1.0000
 Mean   : 6.526   Mean   :0.02877   Mean   :2.997   Mean   :0.6836
 3rd Qu.:10.000   3rd Qu.:0.00000   3rd Qu.:5.000   3rd Qu.:1.0000
 Max.   :12.000   Max.   :1.00000   Max.   :6.000   Max.   :1.0000
   weathersit         temp            atemp             hum
 Min.   :1.000   Min.   : 2.424   Min.   : 3.953   Min.   : 0.00
 1st Qu.:1.000   1st Qu.:13.812   1st Qu.:16.890   1st Qu.:52.00
```

```
 Median :1.000   Median :20.466   Median :24.368   Median :62.62
 Mean   :1.395   Mean   :20.319   Mean   :23.726   Mean   :62.77
 3rd Qu.:2.000   3rd Qu.:26.881   3rd Qu.:30.446   3rd Qu.:72.99
 Max.   :3.000   Max.   :35.328   Max.   :42.045   Max.   :97.25
   windspeed          casual         registered        cnt
 Min.   : 1.500   Min.   :   2.0   Min.   :  20   Min.   :  22
 1st Qu.: 9.042   1st Qu.: 316.2   1st Qu.:2502   1st Qu.:3170
 Median :12.125   Median : 717.0   Median :3664   Median :4548
 Mean   :12.764   Mean   : 849.2   Mean   :3659   Mean   :4508
 3rd Qu.:15.626   3rd Qu.:1096.5   3rd Qu.:4783   3rd Qu.:5966
 Max.   :34.000   Max.   :3410.0   Max.   :6946   Max.   :8714
```

instant 0 **dteday** 0 **season** 0 **yr** 0 **mnth** 0 **holiday** 0 **weekday** 0 **workingday** 0 **weathersit** 0 **temp** 0 **atemp** 0 **hum** 0 **windspeed** 0 **casual** 0 **registered** 0 **cnt** 0

```
Rows: 730
Columns: 9
$ dteday     <date> 2018-01-01, 2018-01-02,

2018-01-03, 2018-01-04, 2018-01-05…
$ cnt        <dbl> 985, 801, 1349,

1562, 1600, 1606, 1510, 959,

822, 1321, 126…
$ season     <fct> Spring, Spring,

Spring, Spring, Spring, Spring,

Spring, Spr…
$ yr         <fct> 2018, 2018, 2018,

2018, 2018, 2018, 2018, 2018,

2018, 2018,…
$ hum        <dbl> 80.5833, 69.6087,

43.7273, 59.0435, 43.6957, 51.8261,

49.86…
$ windspeed  <dbl> 10.749882, 16.652113,

16.636703, 10.739832, 12.522300, 6.00…
$ workingday <fct> No, No, Yes,

Yes, Yes, Yes, Yes, No,

No, Yes, Yes, Yes, Yes…
$ holiday    <fct> No, No, No,

No, No, No, No, No,

No, No, No, No, No,

No, No,…
```

```
$ temp       <dbl> 14.110847, 14.902598,
8.050924, 8.200000, 9.305237, 8.37826…
```

```
     dteday                cnt          season       yr           hum
 Min.   :2018-01-01   Min.   :  22   Spring:180   2018:365   Min.   : 0.00
 1st Qu.:2018-07-02   1st Qu.:3170   Summer:184   2019:365   1st Qu.:52.00
 Median :2018-12-31   Median :4548   Fall  :188              Median :62.62
 Mean   :2018-12-31   Mean   :4508   Winter:178              Mean   :62.77
 3rd Qu.:2019-07-01   3rd Qu.:5966                           3rd Qu.:72.99
 Max.   :2019-12-31   Max.   :8714                           Max.   :97.25
   windspeed      workingday holiday        temp
 Min.   : 1.500   No :231    No :709   Min.   : 2.424
 1st Qu.: 9.042   Yes:499    Yes: 21   1st Qu.:13.812
 Median :12.125                        Median :20.466
 Mean   :12.764                        Mean   :20.319
 3rd Qu.:15.626                        3rd Qu.:26.881
 Max.   :34.000                        Max.   :35.328
```

0

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
     22    3170    4548    4508    5966    8714
```

---

2. Exploratory Data Analysis

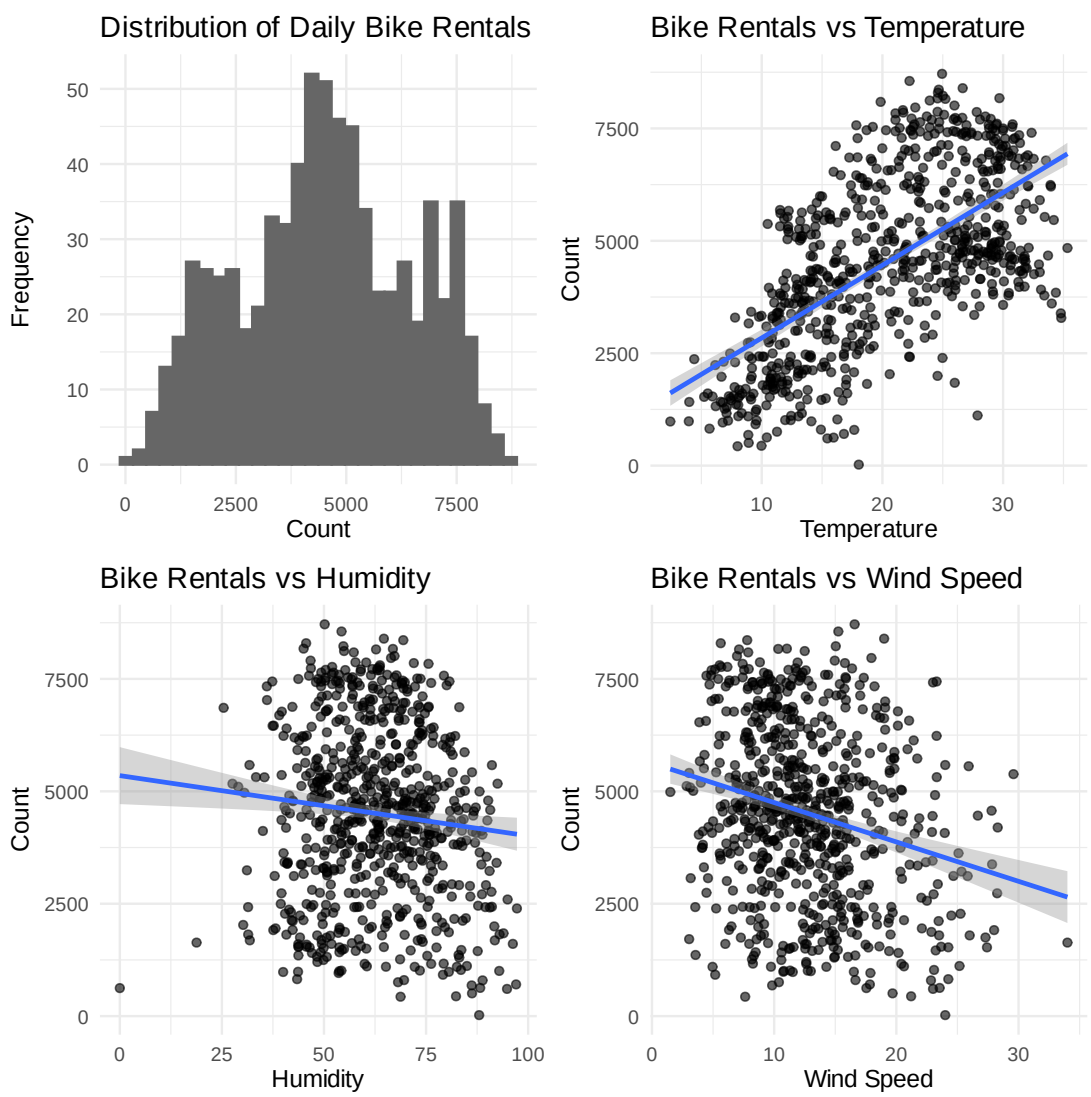
[55]:
```r
# Load required libraries
library(ggplot2)
library(patchwork)

# Figure 1: Histogram of daily bike rentals
p1 <- ggplot(bike_clean, aes(x = cnt)) +
  geom_histogram(bins = 30, fill = "gray40", color = "gray40") +
  labs(
    title = "Distribution of Daily Bike Rentals",
    x = "Count",
    y = "Frequency"
  ) +
  theme_minimal()

# Figure 2: Bike rentals vs temperature
p2 <- ggplot(bike_clean, aes(x = temp, y = cnt)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = TRUE) +
  labs(
    title = "Bike Rentals vs Temperature",
    x = "Temperature",
    y = "Count"
```

```r
  ) +
  theme_minimal()

# Figure 3: Bike rentals vs humidity
p3 <- ggplot(bike_clean, aes(x = hum, y = cnt)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = TRUE) +
  labs(
    title = "Bike Rentals vs Humidity",
    x = "Humidity",
    y = "Count"
  ) +
  theme_minimal()

# Figure 4: Bike rentals vs wind speed
p4 <- ggplot(bike_clean, aes(x = windspeed, y = cnt)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = TRUE) +
  labs(
    title = "Bike Rentals vs Wind Speed",
    x = "Wind Speed",
    y = "Count"
  ) +
  theme_minimal()

# Figure 5: Bike rentals by season
p5 <- ggplot(bike_clean, aes(x = season, y = cnt)) +
  geom_boxplot() +
  labs(
    title = "Bike Rentals by Season",
    x = "Season",
    y = "Count"
  ) +
  theme_minimal()

# Figure 6: Bike rentals by working day
p6 <- ggplot(bike_clean, aes(x = workingday, y = cnt)) +
  geom_boxplot() +
  labs(
    title = "Bike Rentals: Working Day vs Non-Working Day",
    x = "Working Day",
    y = "Count"
  ) +
  theme_minimal()

# Figure 7: Bike rentals by holiday
p7 <- ggplot(bike_clean, aes(x = holiday, y = cnt)) +
```

```r
  geom_boxplot() +
  labs(
    title = "Bike Rentals: Holiday vs Non-Holiday",
    x = "Holiday",
    y = "Count"
  ) +
  theme_minimal()

# Figure 8: Bike rentals by year
p8 <- ggplot(bike_clean, aes(x = yr, y = cnt)) +
  geom_boxplot() +
  labs(
    title = "Bike Rentals by Year",
    x = "Year",
    y = "Count"
  ) +
  theme_minimal()
```

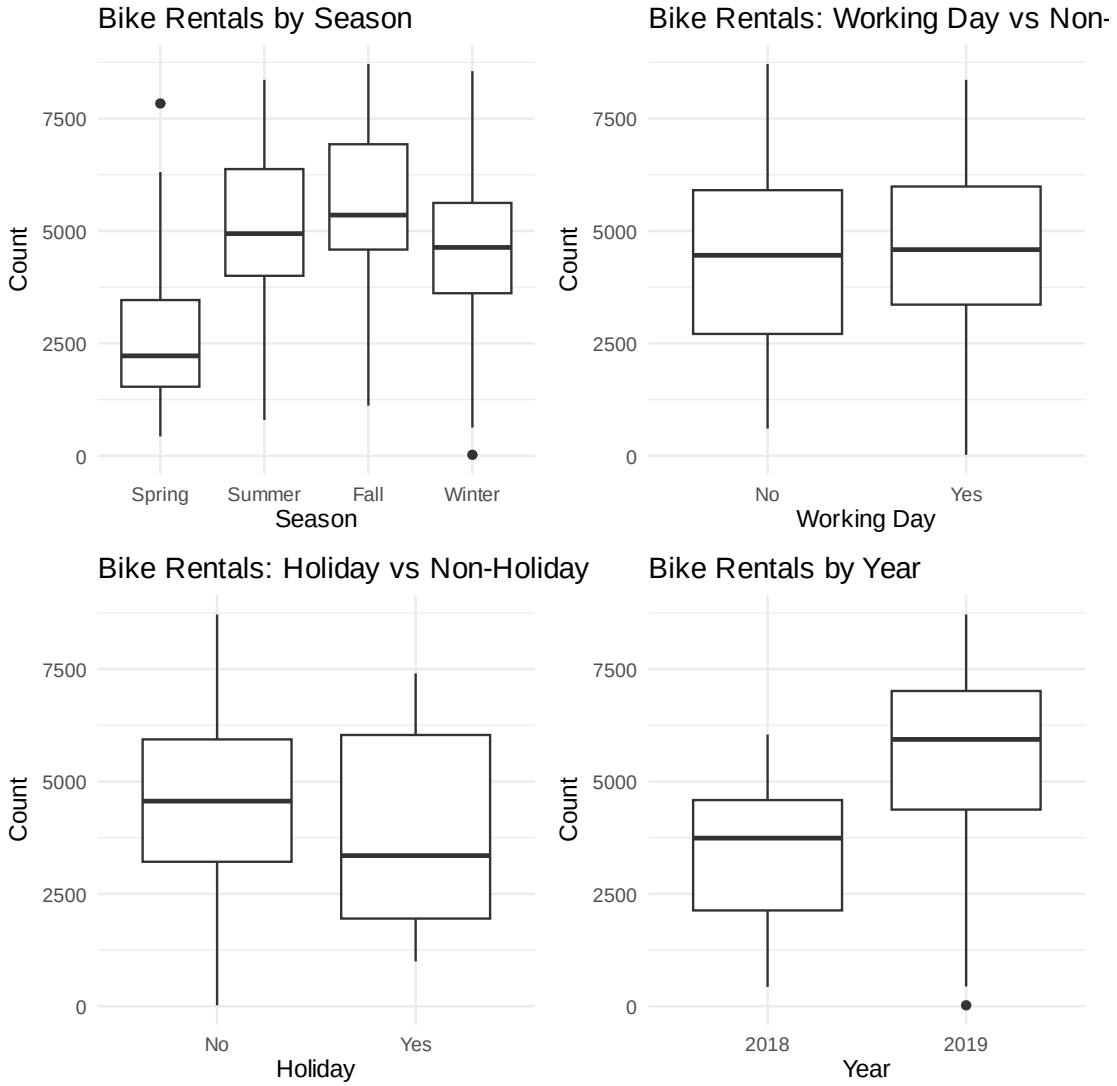
[56]:
```r
# Combine histogram + scatter plots (2x2 layout)
plot_scatter <- (p1 | p2) /
                (p3 | p4)

# Display
plot_scatter

# Save
ggsave(
  filename = "eda_scatter_plots.png",
  plot = plot_scatter,
  width = 12,
  height = 10,
  dpi = 300
)
```

```
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
```



[57]:
```
# Combine boxplots (2x2 layout)
plot_box <- (p5 | p6) /
            (p7 | p8)

# Display
plot_box

# Save
ggsave(
  filename = "eda_boxplots.png",
  plot = plot_box,
  width = 12,
  height = 10,
```

```
  dpi = 300
)
```


Bike Rentals by Season
Count
7500
5000
2500
0
Spring
Summer
Fall
Winter
Season
Bike Rentals: Working Day vs Non-
Count
7500
5000
2500
0
No
Yes
Working Day
Bike Rentals: Holiday vs Non-Holiday
Count
7500
5000
2500
0
No
Yes
Holiday
Bike Rentals by Year
Count
7500
5000
2500
0
2018
2019
Year

[ ]:

```
```

Stat405 Bayesian Coding Part

Aiwei Yang42274209, Chang Sun 19062439

2026-04-17

1. Read and prepare data

```r
# Read dataset
df <- read_csv("day2018.csv", show_col_types = FALSE)

# Convert selected variables to factors
bike_clean <- df %>%
  mutate(
    season = factor(season),
    yr = factor(yr),
    workingday = factor(workingday),
    holiday = factor(holiday)
  )
# Standardize continuous predictors
bike_clean <- bike_clean %>%
  mutate(
    temp_std = as.numeric(scale(temp)),
    hum_std = as.numeric(scale(hum)),
    windspeed_std = as.numeric(scale(windspeed))
  )
# Quick check
glimpse(bike_clean)
```

```
## Rows: 730
## Columns: 19
## $ instant       <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1~
## $ dteday        <chr> "01-01-2018", "02-01-2018", "03-01-2018", "04-01-2018", ~
## $ season        <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ yr            <fct> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ mnth          <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ holiday       <fct> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,~
## $ weekday       <dbl> 6, 0, 1, 2, 3, 4, 5, 6, 0, 1, 2, 3, 4, 5, 6, 0, 1, 2, 3,~
## $ workingday    <fct> 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1,~
## $ weathersit    <dbl> 2, 2, 1, 1, 1, 1, 2, 2, 1, 1, 2, 1, 1, 1, 2, 1, 2, 2, 2,~
## $ temp          <dbl> 14.110847, 14.902598, 8.050924, 8.200000, 9.305237, 8.37~
## $ atemp         <dbl> 18.181250, 17.686950, 9.470250, 10.606100, 11.463500, 11~
## $ hum           <dbl> 80.5833, 69.6087, 43.7273, 59.0435, 43.6957, 51.8261, 49~
## $ windspeed     <dbl> 10.749882, 16.652113, 16.636703, 10.739832, 12.522300, 6~
## $ casual        <dbl> 331, 131, 120, 108, 82, 88, 148, 68, 54, 41, 43, 25, 38,~
## $ registered    <dbl> 654, 670, 1229, 1454, 1518, 1518, 1362, 891, 768, 1280, ~
## $ cnt           <dbl> 985, 801, 1349, 1562, 1600, 1606, 1510, 959, 822, 1321, ~
## $ temp_std      <dbl> -0.8270463, -0.7215741, -1.6343117, -1.6144528, -1.46721~
```

```
## $ hum_std       <dbl> 1.25148472, 0.48066601, -1.33715581, -0.26139783, -1.339~
## $ windspeed_std <dbl> -0.38756724, 0.74838579, 0.74541996, -0.38950148, -0.046~
```

```r
summary(bike_clean)
```

```
##     instant         dteday          season  yr           mnth        holiday
##  Min.   :  1.0   Length:730         1:180   0:365   Min.   : 1.000   0:709
##  1st Qu.:183.2   Class :character   2:184   1:365   1st Qu.: 4.000   1: 21
##  Median :365.5   Mode  :character   3:188           Median : 7.000
##  Mean   :365.5                      4:178           Mean   : 6.526
##  3rd Qu.:547.8                                      3rd Qu.:10.000
##  Max.   :730.0                                      Max.   :12.000
##     weekday      workingday   weathersit         temp            atemp
##  Min.   :0.000   0:231      Min.   :1.000   Min.   : 2.424   Min.   : 3.953
##  1st Qu.:1.000   1:499      1st Qu.:1.000   1st Qu.:13.812   1st Qu.:16.890
##  Median :3.000              Median :1.000   Median :20.466   Median :24.368
##  Mean   :2.997              Mean   :1.395   Mean   :20.319   Mean   :23.726
##  3rd Qu.:5.000              3rd Qu.:2.000   3rd Qu.:26.881   3rd Qu.:30.446
##  Max.   :6.000              Max.   :3.000   Max.   :35.328   Max.   :42.045
##       hum          windspeed          casual         registered
##  Min.   : 0.00   Min.   : 1.500   Min.   :   2.0   Min.   :  20
##  1st Qu.:52.00   1st Qu.: 9.042   1st Qu.: 316.2   1st Qu.:2502
##  Median :62.62   Median :12.125   Median : 717.0   Median :3664
##  Mean   :62.77   Mean   :12.764   Mean   : 849.2   Mean   :3659
##  3rd Qu.:72.99   3rd Qu.:15.626   3rd Qu.:1096.5   3rd Qu.:4783
##  Max.   :97.25   Max.   :34.000   Max.   :3410.0   Max.   :6946
##       cnt          temp_std           hum_std          windspeed_std
##  Min.   :  22   Min.   :-2.38385   Min.   :-4.408413   Min.   :-2.1678
##  1st Qu.:3170   1st Qu.:-0.86687   1st Qu.:-0.756109   1st Qu.:-0.7163
##  Median :4548   Median : 0.01952   Median :-0.009845   Median :-0.1228
##  Mean   :4508   Mean   : 0.00000   Mean   : 0.000000   Mean   : 0.0000
##  3rd Qu.:5966   3rd Qu.: 0.87406   3rd Qu.: 0.718127   3rd Qu.: 0.5508
##  Max.   :8714   Max.   : 1.99942   Max.   : 2.422097   Max.   : 4.0872
```

2. Build design matrix

```r
X <- model.matrix(
  cnt ~ season + yr + temp_std + hum_std + windspeed_std + workingday + holiday,
  data = bike_clean
)
dim(X)
```

```
## [1] 730  10
```

```r
head(X)
```

```
##   (Intercept) season2 season3 season4 yr1   temp_std    hum_std windspeed_std
## 1           1       0       0       0   0 -0.8270463  1.2514847   -0.38756724
## 2           1       0       0       0   0 -0.7215741  0.4806660    0.74838579
## 3           1       0       0       0   0 -1.6343117 -1.3371558    0.74541996
## 4           1       0       0       0   0 -1.6144528 -0.2613978   -0.38950148
## 5           1       0       0       0   0 -1.4672199 -1.3393753   -0.04644478
```

```
## 6           1       0       0       0   0 -1.5907050 -0.7683235   -1.30157017
##   workingday1 holiday1
## 1           0        0
## 2           0        0
## 3           1        0
## 4           1        0
## 5           1        0
## 6           1        0
```

3. Prepare data for Stan

```r
stan_data <- list(
  N = nrow(X),
  K = ncol(X),
  y = bike_clean$cnt,
  X = X
)

stan_data$N
```

```
## [1] 730
```

```r
stan_data$K
```

```
## [1] 10
```

```r
head(stan_data$y)
```

```
## [1]  985  801 1349 1562 1600 1606
```

4. Remove old stan file if they exist

```r
if (file.exists("poisson_model.stan")) file.remove("poisson_model.stan")
if (file.exists("negbin_model.stan")) file.remove("negbin_model.stan")
```

5. Write Poisson Stan model

```r
poisson_code <- "
data {
  int<lower=1> N;
  array[N] int<lower=0> y;
  int<lower=1> K;
  matrix[N, K] X;
}
parameters {
  vector[K] beta;
}
model {
  beta ~ normal(0, 2.5);
  y ~ poisson_log(X * beta);
```

```
}
generated quantities {
  vector[N] log_lik;
  array[N] int y_rep;

  for (n in 1:N) {
    log_lik[n] = poisson_log_lpmf(y[n] | X[n] * beta);
    y_rep[n] = poisson_log_rng(X[n] * beta);
  }
}
"

writeLines(poisson_code, "poisson_model.stan")

# Verify the file contents
readLines("poisson_model.stan")
```

```
##  [1] ""
##  [2] "data {"
##  [3] "  int<lower=1> N;"
##  [4] "  array[N] int<lower=0> y;"
##  [5] "  int<lower=1> K;"
##  [6] "  matrix[N, K] X;"
##  [7] "}"
##  [8] "parameters {"
##  [9] "  vector[K] beta;"
## [10] "}"
## [11] "model {"
## [12] "  beta ~ normal(0, 2.5);"
## [13] "  y ~ poisson_log(X * beta);"
## [14] "}"
## [15] "generated quantities {"
## [16] "  vector[N] log_lik;"
## [17] "  array[N] int y_rep;"
## [18] ""
## [19] "  for (n in 1:N) {"
## [20] "    log_lik[n] = poisson_log_lpmf(y[n] | X[n] * beta);"
## [21] "    y_rep[n] = poisson_log_rng(X[n] * beta);"
## [22] "  }"
## [23] "}"
## [24] ""
```

6. Compile Poisson model

```
poisson_mod <- cmdstan_model("poisson_model.stan")
```

7. Fit Poisson model using HMC

```
fit_poisson_hmc <- poisson_mod$sample(
  data = stan_data,
  chains = 4,
  parallel_chains = 4,
```

```r
  iter_warmup = 1500,
  iter_sampling = 1500,
  adapt_delta = 0.99,
  max_treedepth = 15,
  seed = 123,
  refresh = 500
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 2 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 3 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 4 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 1 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 2 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 1 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 3 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 4 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 2 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 3 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 1 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 4 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 2 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 3 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 4 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 1 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 2 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 3 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 2 finished in 1.5 seconds.
## Chain 3 finished in 1.5 seconds.
## Chain 4 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 4 finished in 1.6 seconds.
## Chain 1 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 1 finished in 1.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 1.6 seconds.
## Total execution time: 1.8 seconds.
```

```r
poisson_hmc_summary <- fit_poisson_hmc$summary()
poisson_hmc_summary
```

```
## # A tibble: 1,471 x 10
##    variable       mean   median      sd     mad       q5      q95  rhat ess_bulk
##    <chr>         <dbl>    <dbl>   <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>
##  1 lp__        2.47e+7  2.47e+7 2.25e+0 1.48e+0  2.47e+7  2.47e+7 1.00     2421.
##  2 beta[1]     7.84e+0  7.84e+0 2.19e-3 2.16e-3  7.84e+0  7.84e+0 1.00     1716.
##  3 beta[2]     3.32e-1  3.32e-1 2.33e-3 2.30e-3  3.28e-1  3.35e-1 1.00     1561.
##  4 beta[3]     2.28e-1  2.28e-1 2.91e-3 2.86e-3  2.23e-1  2.33e-1 1.00     1529.
##  5 beta[4]     4.52e-1  4.52e-1 2.02e-3 2.01e-3  4.49e-1  4.55e-1 1.00     1904.
##  6 beta[5]     4.60e-1  4.60e-1 1.14e-3 1.16e-3  4.58e-1  4.61e-1 1.00     5116.
##  7 beta[6]     2.43e-1  2.43e-1 1.04e-3 1.00e-3  2.41e-1  2.45e-1 1.00     2472.
##  8 beta[7]    -8.70e-2 -8.69e-2 6.19e-4 6.18e-4 -8.80e-2 -8.59e-2 1.00     8224.
##  9 beta[8]    -6.24e-2 -6.24e-2 6.12e-4 6.17e-4 -6.34e-2 -6.14e-2 1.000    8887.
## 10 beta[9]     1.87e-2  1.87e-2 1.24e-3 1.25e-3  1.67e-2  2.08e-2 1.00     4467.
## # i 1,461 more rows
## # i 1 more variable: ess_tail <dbl>
```

8. Inspect Poisson HMC diagnostics

```r
# Print summary table
poisson_hmc_summary
```

```
## # A tibble: 1,471 x 10
##    variable       mean   median      sd     mad       q5      q95  rhat ess_bulk
##    <chr>         <dbl>    <dbl>   <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>
##  1 lp__        2.47e+7  2.47e+7 2.25e+0 1.48e+0  2.47e+7  2.47e+7 1.00     2421.
##  2 beta[1]     7.84e+0  7.84e+0 2.19e-3 2.16e-3  7.84e+0  7.84e+0 1.00     1716.
##  3 beta[2]     3.32e-1  3.32e-1 2.33e-3 2.30e-3  3.28e-1  3.35e-1 1.00     1561.
##  4 beta[3]     2.28e-1  2.28e-1 2.91e-3 2.86e-3  2.23e-1  2.33e-1 1.00     1529.
##  5 beta[4]     4.52e-1  4.52e-1 2.02e-3 2.01e-3  4.49e-1  4.55e-1 1.00     1904.
##  6 beta[5]     4.60e-1  4.60e-1 1.14e-3 1.16e-3  4.58e-1  4.61e-1 1.00     5116.
##  7 beta[6]     2.43e-1  2.43e-1 1.04e-3 1.00e-3  2.41e-1  2.45e-1 1.00     2472.
##  8 beta[7]    -8.70e-2 -8.69e-2 6.19e-4 6.18e-4 -8.80e-2 -8.59e-2 1.00     8224.
##  9 beta[8]    -6.24e-2 -6.24e-2 6.12e-4 6.17e-4 -6.34e-2 -6.14e-2 1.000    8887.
## 10 beta[9]     1.87e-2  1.87e-2 1.24e-3 1.25e-3  1.67e-2  2.08e-2 1.00     4467.
## # i 1,461 more rows
## # i 1 more variable: ess_tail <dbl>
```

```r
# Check the largest R-hat value
max(poisson_hmc_summary$rhat, na.rm = TRUE)
```

```
## [1] 1.003341
```

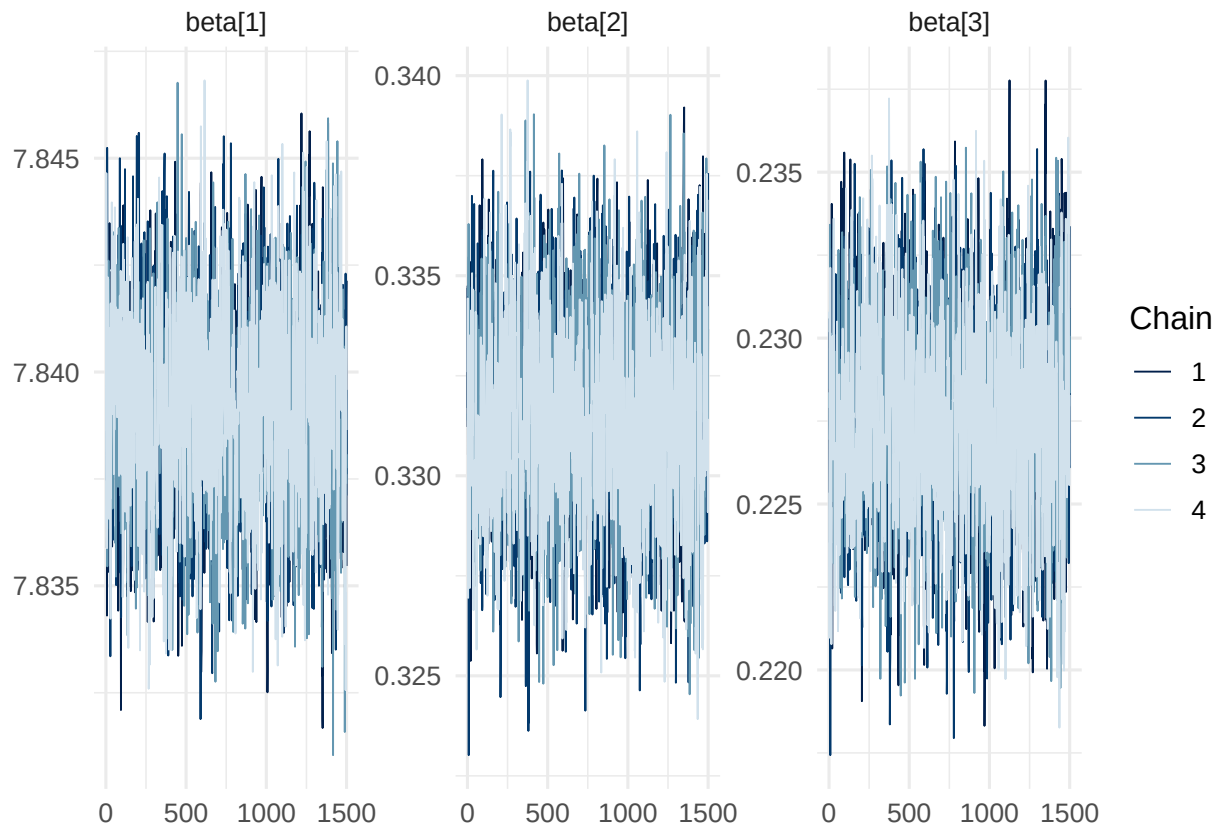
```r
# Check the smallest bulk ESS
min(poisson_hmc_summary$ess_bulk, na.rm = TRUE)
```

```
## [1] 1529.346
```

```r
# Check the smallest tail ESS
min(poisson_hmc_summary$ess_tail, na.rm = TRUE)
```

```
## [1] 1831.3
```

```r
# Traceplots for selected Poisson coefficients
mcmc_trace(
  fit_poisson_hmc$draws(variables = c("beta[1]", "beta[2]", "beta[3]"))
)
```



9. Fit Poisson model using Variational Inference

```r
fit_poisson_vi <- poisson_mod$variational(
  data = stan_data,
  algorithm = "meanfield",
  iter = 20000,
  output_samples = 2000,
  seed = 123
)
```

```
## ------------------------------------------------------------
## EXPERIMENTAL ALGORITHM:
##   This procedure has not been thoroughly tested and may be unstable
##   or buggy. The interface is subject to change.
## ------------------------------------------------------------
## Gradient evaluation took 1.8e-05 seconds
## 1000 transitions using 10 leapfrog steps per transition would take 0.18 seconds.
## Adjust your expectations accordingly!
## Begin eta adaptation.
```

```
## Iteration:   1 / 250 [  0%]  (Adaptation)
## Iteration:  50 / 250 [ 20%]  (Adaptation)
## Iteration: 100 / 250 [ 40%]  (Adaptation)
## Iteration: 150 / 250 [ 60%]  (Adaptation)
## Iteration: 200 / 250 [ 80%]  (Adaptation)
## Success! Found best value [eta = 1] earlier than expected.
## Begin stochastic gradient ascent.
##   iter             ELBO   delta_ELBO_mean   delta_ELBO_med   notes
##    100     -2687516.618             1.000            1.000
##    200      -446077.710             3.012            5.025
##    300      -101285.998             3.143            3.404
##    400      -112202.212             2.382            3.404
##    500       -87860.669             1.961            1.000
##    600       -82139.872             1.645            1.000
##    700       -82847.598             1.412            0.277
##    800       -94800.546             1.251            0.277
##    900       -89769.916             1.118            0.126
##   1000       -86413.230             1.010            0.126
##   1100       -89462.231             0.921            0.097   MAY BE DIVERGING... INSPECT ELBO
##   1200       -86232.827             0.848            0.097   MAY BE DIVERGING... INSPECT ELBO
##   1300       -87128.074             0.783            0.070   MAY BE DIVERGING... INSPECT ELBO
##   1400       -87795.030             0.728            0.070   MAY BE DIVERGING... INSPECT ELBO
##   1500       -86619.860             0.680            0.056   MAY BE DIVERGING... INSPECT ELBO
##   1600       -84381.965             0.639            0.056   MAY BE DIVERGING... INSPECT ELBO
##   1700       -88291.397             0.604            0.044   MAY BE DIVERGING... INSPECT ELBO
##   1800       -85620.870             0.573            0.044   MAY BE DIVERGING... INSPECT ELBO
##   1900       -82841.597             0.544            0.039   MAY BE DIVERGING... INSPECT ELBO
##   2000       -88853.759             0.520            0.044   MAY BE DIVERGING... INSPECT ELBO
##   2100       -83242.743             0.474            0.044
##   2200       -84042.407             0.223            0.039
##   2300       -84309.521             0.053            0.037
##   2400       -84856.001             0.048            0.034
##   2500       -82544.820             0.036            0.034
##   2600       -83741.473             0.033            0.031
##   2700       -82740.379             0.033            0.031
##   2800       -84308.846             0.028            0.028
##   2900       -84333.718             0.025            0.027
##   3000       -83939.880             0.024            0.019
##   3100       -84139.269             0.022            0.014
##   3200       -82799.150             0.021            0.014
##   3300       -83281.625             0.021            0.014
##   3400       -83891.623             0.021            0.014
##   3500       -83244.158             0.020            0.014
##   3600       -82497.327             0.019            0.012
##   3700       -82496.565             0.017            0.010   MEDIAN ELBO CONVERGED
## Drawing a sample of size 2000 from the approximate posterior...
## COMPLETED.
## Finished in  0.8 seconds.
```

```r
# Extract summary
poisson_vi_summary <- fit_poisson_vi$summary()

# Print summary table
poisson_vi_summary
```

```
## # A tibble: 1,472 x 7
##    variable           mean      median         sd        mad         q5      q95
##    <chr>             <dbl>       <dbl>      <dbl>      <dbl>      <dbl>    <dbl>
##  1 lp__         -82446.     -82441.     141.       136.         -8.27e+4 -8.22e+4
##  2 lp_approx__     -4.97       -4.64     2.23       2.11       -9.13e+0 -1.97e+0
##  3 beta[1]          7.83        7.83     0.000984   0.000980    7.83e+0  7.83e+0
##  4 beta[2]          0.323       0.323    0.00129    0.00132     3.21e-1  3.26e-1
##  5 beta[3]          0.220       0.220    0.000691   0.000688    2.19e-1  2.22e-1
##  6 beta[4]          0.445       0.445    0.00183    0.00189     4.42e-1  4.48e-1
##  7 beta[5]          0.452       0.452    0.000375   0.000365    4.51e-1  4.53e-1
##  8 beta[6]          0.235       0.235    0.000543   0.000542    2.34e-1  2.36e-1
##  9 beta[7]         -0.0958     -0.0958   0.000517   0.000499   -9.66e-2 -9.49e-2
## 10 beta[8]         -0.0559     -0.0560   0.00113    0.00113    -5.78e-2 -5.41e-2
## # i 1,462 more rows
```

10. Compare Poisson HMC and VI

```r
poisson_hmc_beta <- fit_poisson_hmc$summary(variables = "beta")
poisson_vi_beta  <- fit_poisson_vi$summary(variables = "beta")

comparison_poisson <- data.frame(
  parameter = poisson_hmc_beta$variable,
  hmc_mean  = poisson_hmc_beta$mean,
  hmc_sd    = poisson_hmc_beta$sd,
  vi_mean   = poisson_vi_beta$mean,
  vi_sd     = poisson_vi_beta$sd
)

comparison_poisson
```

```
##    parameter    hmc_mean       hmc_sd     vi_mean        vi_sd
## 1    beta[1]  7.83911134 0.0021856435  7.83167741 0.0009839892
## 2    beta[2]  0.33154175 0.0023290504  0.32348294 0.0012929324
## 3    beta[3]  0.22769814 0.0029090617  0.22046135 0.0006911088
## 4    beta[4]  0.45176851 0.0020161697  0.44501448 0.0018317399
## 5    beta[5]  0.45962165 0.0011433077  0.45202753 0.0003753108
## 6    beta[6]  0.24295421 0.0010411229  0.23491647 0.0005425597
## 7    beta[7] -0.08695160 0.0006188522 -0.09578930 0.0005170358
## 8    beta[8] -0.06241207 0.0006118849 -0.05594871 0.0011308019
## 9    beta[9]  0.01873114 0.0012395911  0.01156136 0.0011505168
## 10  beta[10] -0.14658813 0.0037096590 -0.15317263 0.0032456575
```

11. Write Negative Binomial Stan model

```r
negbin_code <- "
data {
  int<lower=1> N;
  array[N] int<lower=0> y;
  int<lower=1> K;
  matrix[N, K] X;
}
parameters {
```

```
  vector[K] beta;
  real log_phi;
}
transformed parameters {
  real<lower=0> phi;
  phi = exp(log_phi);
}
model {
  beta ~ normal(0, 2.5);
  log_phi ~ normal(0, 1);
  y ~ neg_binomial_2_log(X * beta, phi);
}
generated quantities {
  vector[N] log_lik;
  array[N] int y_rep;
  for (n in 1:N) {
    log_lik[n] = neg_binomial_2_log_lpmf(y[n] | X[n] * beta, phi);
    y_rep[n] = neg_binomial_2_log_rng(X[n] * beta, phi);
  }
}
"

# Remove old file if it exists
if (file.exists("negbin_model.stan")) file.remove("negbin_model.stan")

# Write the Stan file
writeLines(negbin_code, "negbin_model.stan")

# Verify file contents
readLines("negbin_model.stan")
```

```
##  [1] ""
##  [2] "data {"
##  [3] "  int<lower=1> N;"
##  [4] "  array[N] int<lower=0> y;"
##  [5] "  int<lower=1> K;"
##  [6] "  matrix[N, K] X;"
##  [7] "}"
##  [8] "parameters {"
##  [9] "  vector[K] beta;"
## [10] "  real log_phi;"
## [11] "}"
## [12] "transformed parameters {"
## [13] "  real<lower=0> phi;"
## [14] "  phi = exp(log_phi);"
## [15] "}"
## [16] "model {"
## [17] "  beta ~ normal(0, 2.5);"
## [18] "  log_phi ~ normal(0, 1);"
## [19] "  y ~ neg_binomial_2_log(X * beta, phi);"
## [20] "}"
## [21] "generated quantities {"
## [22] "  vector[N] log_lik;"
```

```
## [23] "  array[N] int y_rep;"
## [24] "  for (n in 1:N) {"
## [25] "    log_lik[n] = neg_binomial_2_log_lpmf(y[n] | X[n] * beta, phi);"
## [26] "    y_rep[n] = neg_binomial_2_log_rng(X[n] * beta, phi);"
## [27] "  }"
## [28] "}"
## [29] ""
```

12. Compile Negative Binomial model

```r
nb_mod <- cmdstan_model("negbin_model.stan", force_recompile = TRUE)
```

13. Fit Negative Binomial model using HMC

```r
fit_nb_hmc <- nb_mod$sample(
  data = stan_data,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1500,
  iter_sampling = 1500,
  adapt_delta = 0.99,
  max_treedepth = 15,
  seed = 123,
  refresh = 500
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 3000 [  0%]  (Warmup)

## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 1

## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 1
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
```

```
## Chain 1

## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 1 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 1

## Chain 2 Iteration:    1 / 3000 [  0%]  (Warmup)

## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 2

## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 2

## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 2

## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
```

```
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 3 Iteration:    1 / 3000 [  0%]  (Warmup)
```

```
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
```

```
## Chain 3

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 3

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 3

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 3 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 3

## Chain 4 Iteration:    1 / 3000 [  0%]  (Warmup)

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 4

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
```

```
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
```

```
## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 4

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of th

## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 4

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of th

## Chain 4 Exception: neg_binomial_2_log_lpmf: Precision parameter is 0, but must be positive finite! (

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 4

## Chain 4 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 1 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 2 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 3 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 4 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 1 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 2 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 3 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 4 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1500 / 3000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1501 / 3000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 2 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 1 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 3 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 4 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 2 Iteration: 2500 / 3000 [ 83%]  (Sampling)
```

```
## Chain 1 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 4 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 4 finished in 4.4 seconds.
## Chain 2 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 2 finished in 4.7 seconds.
## Chain 1 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 3 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 1 finished in 4.9 seconds.
## Chain 3 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 3 finished in 6.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 5.0 seconds.
## Total execution time: 6.0 seconds.
```

```r
# Extract summary
nb_hmc_summary <- fit_nb_hmc$summary()

# Print summary table
nb_hmc_summary
```

```
## # A tibble: 1,473 x 10
##    variable       mean     median     sd    mad       q5      q95  rhat ess_bulk
##    <chr>         <dbl>      <dbl>  <dbl>  <dbl>    <dbl>    <dbl> <dbl>    <dbl>
##  1 lp__      -6141.     -6141.     2.40   2.20   -6.15e+3 -6.14e+3  1.00    2399.
##  2 beta[1]      7.83       7.83   0.0347 0.0336  7.77e+0  7.88e+0  1.00    1899.
##  3 beta[2]      0.309      0.308  0.0365 0.0365  2.50e-1  3.70e-1  1.00    2333.
##  4 beta[3]      0.177      0.177  0.0484 0.0485  9.55e-2  2.56e-1  1.00    2072.
##  5 beta[4]      0.464      0.464  0.0319 0.0321  4.12e-1  5.17e-1  1.00    3216.
##  6 beta[5]      0.482      0.482  0.0201 0.0207  4.48e-1  5.15e-1  1.00    5223.
##  7 beta[6]      0.304      0.304  0.0185 0.0184  2.74e-1  3.34e-1  1.00    2416.
##  8 beta[7]     -0.106     -0.106  0.0110 0.0110 -1.24e-1 -8.76e-2  1.00    5632.
##  9 beta[8]     -0.0745    -0.0745 0.0107 0.0107 -9.19e-2 -5.73e-2  1.00    5755.
## 10 beta[9]      0.0328     0.0329 0.0218 0.0220 -2.65e-3  6.88e-2  1.00    4491.
## # i 1,463 more rows
## # i 1 more variable: ess_tail <dbl>
```

Since there is warning, we need to check r-hat and ESS

```r
nb_hmc_summary <- fit_nb_hmc$summary()

max(nb_hmc_summary$rhat)
```

```
## [1] 1.004275
```

```r
min(nb_hmc_summary$ess_bulk)
```

```
## [1] 1899.109
```

14. Inspect Negative Binomial HMC diagnostics

```r
# Check the largest R-hat value
max(nb_hmc_summary$rhat, na.rm = TRUE)
```

```
## [1] 1.004275
```

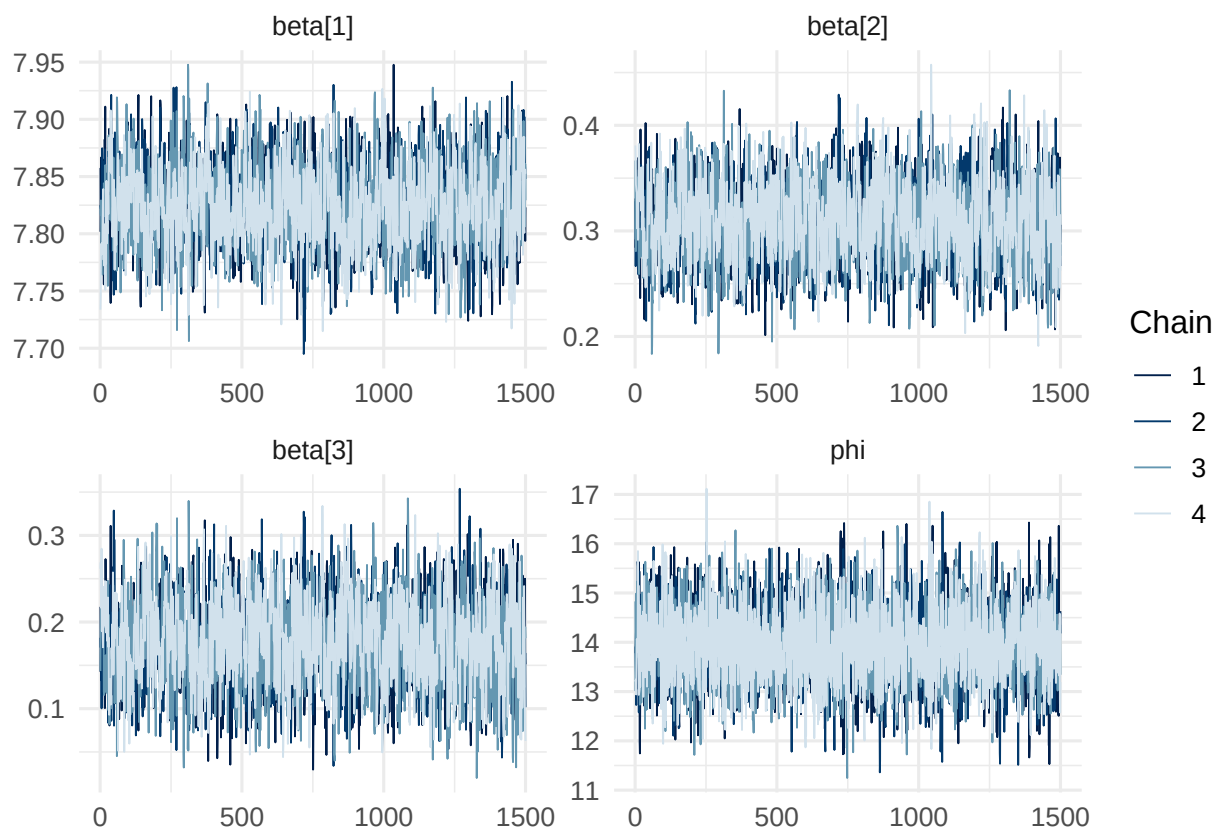
```r
# Check the smallest bulk ESS
min(nb_hmc_summary$ess_bulk, na.rm = TRUE)
```

```
## [1] 1899.109
```

```r
# Check the smallest tail ESS
min(nb_hmc_summary$ess_tail, na.rm = TRUE)
```

```
## [1] 2758.454
```

```r
# Traceplots for selected coefficients and dispersion parameter
mcmc_trace(
  fit_nb_hmc$draws(variables = c("beta[1]", "beta[2]", "beta[3]", "phi"))
)
```



15. Fit Negative Binomial model using Variational Inference

```r
fit_nb_vi <- nb_mod$variational(
  data = stan_data,
  algorithm = "meanfield",
  iter = 20000,
  output_samples = 2000,
  seed = 123
)
```

```
## ------------------------------------------------------------
## EXPERIMENTAL ALGORITHM:
##   This procedure has not been thoroughly tested and may be unstable
##   or buggy. The interface is subject to change.
## ------------------------------------------------------------
## Gradient evaluation took 5.5e-05 seconds
## 1000 transitions using 10 leapfrog steps per transition would take 0.55 seconds.
## Adjust your expectations accordingly!
## Begin eta adaptation.
## Iteration:   1 / 250 [  0%]  (Adaptation)
## Iteration:  50 / 250 [ 20%]  (Adaptation)
## Iteration: 100 / 250 [ 40%]  (Adaptation)
## Iteration: 150 / 250 [ 60%]  (Adaptation)
## Iteration: 200 / 250 [ 80%]  (Adaptation)
## Success! Found best value [eta = 1] earlier than expected.
## Begin stochastic gradient ascent.
##   iter             ELBO   delta_ELBO_mean   delta_ELBO_med   notes
##    100        -9180.183             1.000            1.000
##    200        -8671.916             0.529            1.000
##    300        -7988.055             0.381            0.086
##    400        -6902.279             0.325            0.157
##    500        -6356.607             0.277            0.086
##    600        -6233.192             0.235            0.086
##    700        -6194.169             0.202            0.086
##    800        -6204.706             0.177            0.086
##    900        -6192.126             0.157            0.059
##   1000        -6211.608             0.142            0.059
##   1100        -6201.880             0.129            0.020
##   1200        -6197.756             0.119            0.020
##   1300        -6233.682             0.110            0.006   MEDIAN ELBO CONVERGED
## Drawing a sample of size 2000 from the approximate posterior...
## COMPLETED.
## Finished in  1.0 seconds.
```

```r
# Extract summary
nb_vi_summary <- fit_nb_vi$summary()

# Print summary table
nb_vi_summary
```

```
## # A tibble: 1,474 x 7
##    variable          mean     median       sd      mad         q5        q95
##    <chr>            <dbl>      <dbl>    <dbl>    <dbl>      <dbl>      <dbl>
##  1 lp__         -6205.     -6203.     15.1     14.6     -6232.     -6182.
##  2 lp_approx__    -5.43      -5.11     2.32     2.13       -9.60      -2.31
```

```
##  3 beta[1]         7.85       7.85    0.00913  0.00912     7.84       7.87
##  4 beta[2]         0.333      0.333   0.0164   0.0164      0.307      0.361
##  5 beta[3]         0.208      0.208   0.0219   0.0215      0.172      0.244
##  6 beta[4]         0.501      0.502   0.0222   0.0218      0.465      0.538
##  7 beta[5]         0.507      0.507   0.0142   0.0142      0.483      0.530
##  8 beta[6]         0.332      0.332   0.0124   0.0125      0.312      0.353
##  9 beta[7]        -0.119     -0.119   0.0136   0.0137     -0.141     -0.0965
## 10 beta[8]        -0.0661    -0.0663  0.0109   0.0108     -0.0839    -0.0484
## # i 1,464 more rows
```

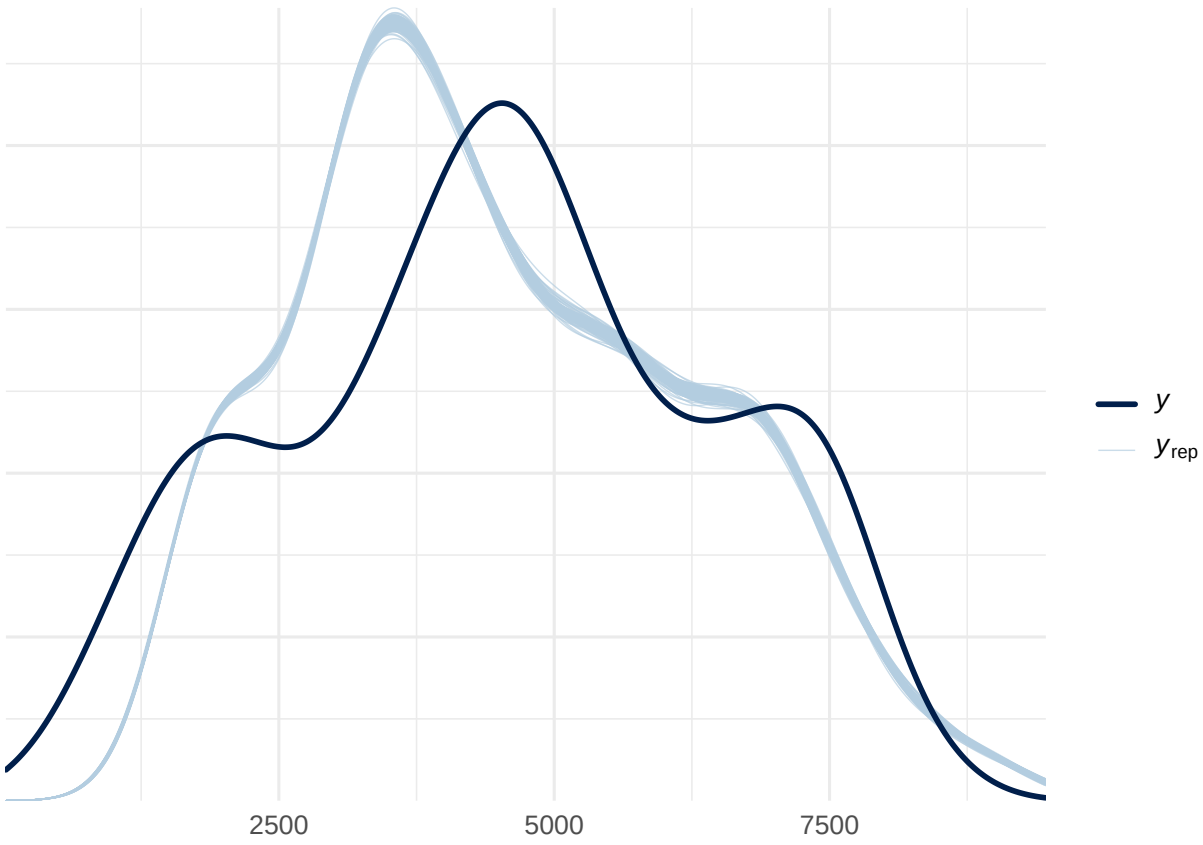
16. Compare Negative Binomial HMC and VI

```r
nb_hmc_beta <- fit_nb_hmc$summary(variables = c("beta", "phi"))
nb_vi_beta  <- fit_nb_vi$summary(variables = c("beta", "phi"))

comparison_nb <- data.frame(
  parameter = nb_hmc_beta$variable,
  hmc_mean  = nb_hmc_beta$mean,
  hmc_sd    = nb_hmc_beta$sd,
  vi_mean   = nb_vi_beta$mean,
  vi_sd     = nb_vi_beta$sd
)

comparison_nb
```

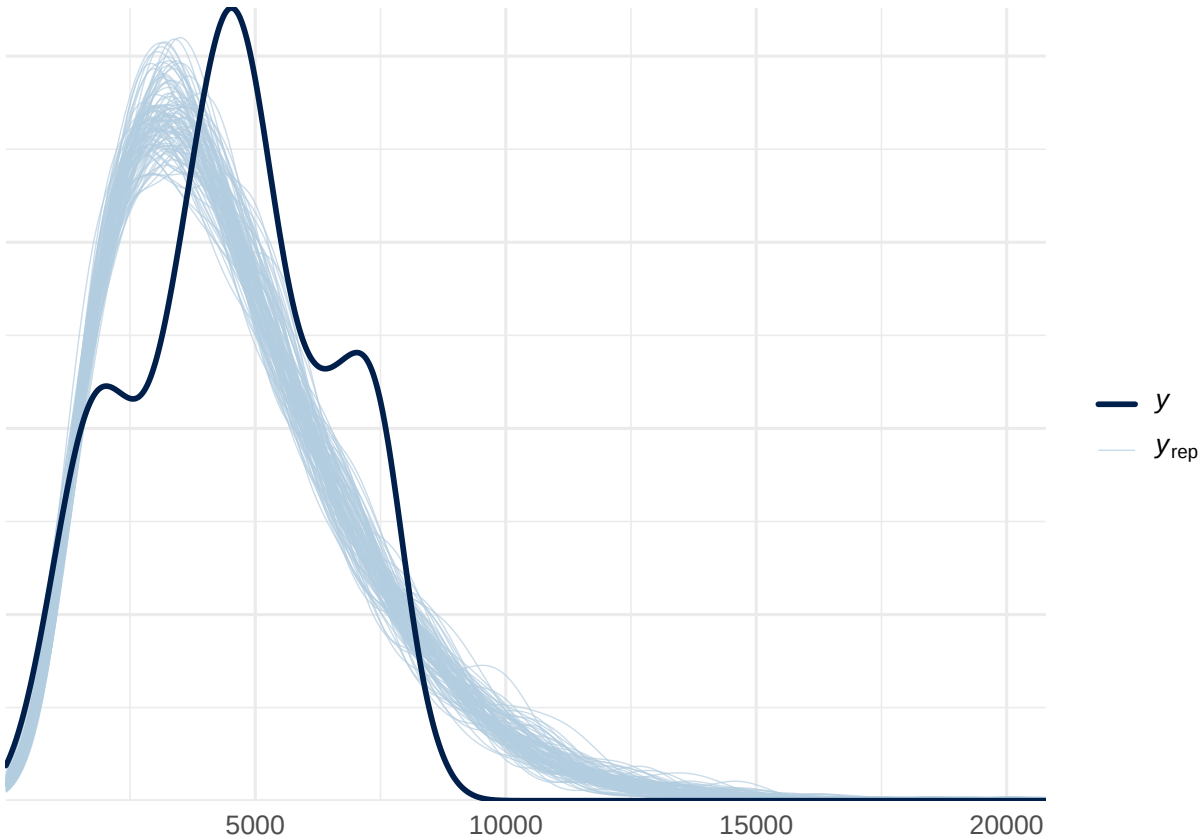
```
##    parameter    hmc_mean     hmc_sd     vi_mean       vi_sd
## 1    beta[1]  7.82624806 0.03474125  7.85464401 0.009132069
## 2    beta[2]  0.30908460 0.03646106  0.33341360 0.016411603
## 3    beta[3]  0.17654453 0.04835608  0.20755368 0.021862086
## 4    beta[4]  0.46387628 0.03186679  0.50145743 0.022186716
## 5    beta[5]  0.48165124 0.02014094  0.50677473 0.014215533
## 6    beta[6]  0.30401928 0.01845978  0.33191739 0.012422683
## 7    beta[7] -0.10581898 0.01104960 -0.11910159 0.013586677
## 8    beta[8] -0.07453996 0.01066716 -0.06613203 0.010944632
## 9    beta[9]  0.03280309 0.02178496  0.06454817 0.010212477
## 10  beta[10] -0.17016123 0.06123285 -0.14979639 0.067730173
## 11       phi 13.94255775 0.74569683 13.38369343 0.708746702
```

17. Posterior predictive checks

```r
# Extract posterior predictive draws
yrep_poisson <- as_draws_matrix(fit_poisson_hmc$draws("y_rep"))
yrep_nb <- as_draws_matrix(fit_nb_hmc$draws("y_rep"))

# Posterior predictive density overlay
ppc_dens_overlay(y = bike_clean$cnt, yrep = yrep_poisson[1:100, ])
```




```
ppc_dens_overlay(y = bike_clean$cnt, yrep = yrep_nb[1:100, ])
```




```r
# Compare observed and replicated mean and variance
ppc_stats <- data.frame(
  model = c("Observed", "Poisson_HMC", "NB_HMC"),
  mean = c(
    mean(bike_clean$cnt),
    mean(rowMeans(yrep_poisson)),
    mean(rowMeans(yrep_nb))
  ),
  variance = c(
    var(bike_clean$cnt),
    mean(apply(yrep_poisson, 1, var)),
    mean(apply(yrep_nb, 1, var))
  )
)

ppc_stats
```

```
##         model     mean variance
## 1    Observed 4508.007  3748141
## 2 Poisson_HMC 4508.043  3143307
## 3      NB_HMC 4555.502  5731131
```

18. Save all summary tables

```r
write.csv(as.data.frame(poisson_hmc_summary), "poisson_hmc_summary.csv", row.names = FALSE)
write.csv(as.data.frame(poisson_vi_summary), "poisson_vi_summary.csv", row.names = FALSE)
write.csv(as.data.frame(nb_hmc_summary), "nb_hmc_summary.csv", row.names = FALSE)
write.csv(as.data.frame(nb_vi_summary), "nb_vi_summary.csv", row.names = FALSE)

write.csv(comparison_poisson, "comparison_poisson.csv", row.names = FALSE)
write.csv(comparison_nb, "comparison_nb.csv", row.names = FALSE)
write.csv(ppc_stats, "ppc_stats.csv", row.names = FALSE)
```

19. Compare computation times

```r
fit_poisson_hmc$time()
```

```
## $total
## [1] 1.809298
##
## $chains
##   chain_id warmup sampling total
## 1        1  0.669    1.005 1.674
## 2        2  0.587    0.929 1.516
## 3        3  0.580    0.912 1.492
## 4        4  0.581    1.028 1.609
```

```r
fit_poisson_vi$time()
```

```
## $total
## [1] 0.7630591
```

```r
fit_nb_hmc$time()
```

```
## $total
## [1] 6.042777
##
## $chains
##   chain_id warmup sampling total
## 1        1  2.307    2.640 4.947
## 2        2  2.231    2.491 4.722
## 3        3  2.502    3.501 6.003
## 4        4  2.011    2.439 4.450
```

```r
fit_nb_vi$time()
```

```
## $total
## [1] 0.9646602
```